12. Lambda Expressions Syntax — Quick Notes

General Syntax

(parameters) -> expression
or
(parameters) -> {
  statements
}

No Parameter Lambda

() -> System.out.println("Hello")

Used for: void print() type methods.

Single Parameter Lambda

p -> System.out.println(p)

Important

For single parameter: brackets optional
Can also write: (p) -> System.out.println(p)

Multiple Parameters Lambda

(p1, p2) -> System.out.println(p1 + " " + p2)

Used when function takes: 2 or more parameters

Important Point

Datatype NOT required.

Compiler automatically infers:

- datatype
- return type

Example: p -> System.out.println(p)

Compiler infers: p is Integer, if the functional interface expects Integer.

Assigning Lambda to Variable

Predicate<Integer> p = x -> x % 2 == 0;

Lambda assigned to:Functional Interface reference

Example — No Parameter

interface Test1 {
  void print();
}
Usage: Test1 t = () -> System.out.println("Hello");

Output:Hello

Example — Single Parameter

```
interface Test2 {
   void print(int p);
}

Usage: Test2 t = p -> System.out.println(p);
          t.print(10);

Output: 10
```

Example — Multiple Parameters

```
interface Test3 {
   void print(int p1, int p2);
}

Usage: Test3 t = (p1, p2) ->System.out.println(p1 + " " + p2);

Output: 10 20
```

Single Line vs Multi Line

Single Line

```
p -> System.out.println(p)
```

No curly brackets needed.

Multi Line

```
p -> {
   if(p % 2 == 0)
      System.out.println("Even");
   else
      System.out.println("Odd");
}
```

Important: Multi-line Lambda:must use {}

Return Type Lambda

Explicit Return

```
p -> {
          return p % 2 == 0;
}
```

Compiler Inferred Return

```
p -> p + 2
```

Compiler automatically infers:

- return type
- return statement

Important Rule

Case	Curly Brackets
Single line	Optional
Multi-line	Mandatory
Using `return`	Mandatory

Sorting Example with Lambda

Arrays.sort(arr,(p1, p2) -> p1.x - p2.x);
Used for: custom sorting
Comparator implemented using Lambda.

Important Quick Points

Topic	Point
Lambda	Anonymous function
`->`	Arrow operator
Datatype	Auto inferred
Return Type	Auto inferred
Single parameter	`()` optional
Multi-line Lambda	`{}` required
Works with	Functional Interfaces
Common use	Streams, sorting, filtering